CHALLENGE | ONG TURMA DO BEM

Raiz do Bem - Plataforma integrada para a Ong Turma do Bem

Documentação – Sprint 3: Domain Driven Design Using Java

MURILO AYABE SEVERINO - RM567479

PAULO CAVALCANTE CAROBA - RM566667

RENAN DA SILVA PAULINO - RM566610

Tecnólogo em Análise e Desenvolvimento de Sistemas

1TDSPS – Profº Fernando Luiz

SÃO PAULO

2026

Sumário

Introdução 1
Descrição geral do projeto 1
Objetivo 1
Justificativa da Solução 1
Funcionalidades implementadas 2
Visualização de telas do sistema 4
Métodos com lógica de negócio 6
Modelo de Entidade Relacionamento – (MER) 9
Arquitetura do Projeto 10
Diagramas de Classes 11
Diagrama de Classes Simples 11
Diagrama UML 11
Procedimentos para execução 12
Pequena demonstração do programa 15
Referências relevantes no projeto 15
Conclusão 15

Total de Páginas: 18

INTRODUÇÃO

A Ong Turma do Bem foi fundada inspirada pelo trabalho de um dentista, autor do livro "Um Sorriso Feliz para o Seu Filho". Durante palestras em escolas, percebeu-se que as famílias de escolas particulares conseguiam aplicar os conselhos de saúde bucal, enquanto em escolas públicas os relatos eram de problemas odontológicos graves e sem condições de resolução. Assim, nasceu uma organização social dedicada a transformar a vida de jovens de baixa renda por meio do acesso gratuito a tratamento odontológico.

Hoje, a Turma do Bem conta com aproximadamente 18,5 mil dentistas voluntários e 320 coordenadores, tendo beneficiado cerca de 90 mil pessoas. Com atuação em 120 municípios brasileiros, a ONG também está presente em 12 países da América Latina e em Portugal, sempre dependente do apoio de patrocinadores para manter suas atividades.

Atualmente, o atendimento e a gestão dos casos ocorrem de forma descentralizada, empregando canais como WhatsApp, e-mail e formulários simples, o que resulta em falhas, atrasos e perda de informações. Diante desse contexto, este documento apresenta a primeira entrega do projeto Raiz do Bem: uma plataforma digital desenvolvida com base nos conceitos de Domain Driven Design, que visa centralizar o atendimento, otimizar fluxos e trazer mais transparência e organização para os processos da ONG Turma do Bem.

DESCRIÇÃO GERAL DO PROJETO

Objetivo

O sistema "Raiz do Bem" foi desenvolvido para gerenciar atendimentos odontológicos prestados por uma ONG, permitindo o cadastro e acompanhamento de beneficiários, dentistas, pedidos de ajuda e atendimentos realizados.

Justificativa da Solução

Considerando os desafios operacionais da ONG Turma do Bem — como informações distribuídas em diferentes canais, processos manuais e baixa rastreabilidade dos atendimentos — a solução proposta evoluiu para uma plataforma estruturada em Java com foco em organização, persistência e confiabilidade dos dados.

Na Sprint 3, o projeto consolidou a arquitetura em camadas (View, Controller, BO e DAO), com integração efetiva ao banco Oracle, permitindo que os processos essenciais tenham persistência real e fluxo consistente de ponta a ponta. A implementação de operações CRUD nos módulos centrais trouxe maior controle sobre o ciclo de vida dos dados, reduziu retrabalho e melhorou a qualidade das informações utilizadas nas decisões da ONG.

A modelagem orientada ao domínio se justifica pela natureza sensível do problema social e de saúde atendido: ela permite representar regras de negócio com maior precisão, separar responsabilidades técnicas e preparar o sistema para evolução contínua sem perda de consistência. Além disso, a adoção de entidades de apoio com catálogo fixo (como especialidades e programas sociais) reforça padronização e integridade no cadastro, evitando inconsistências operacionais.

Com essa evolução, a solução atende melhor aos objetivos de eficiência, transparência e governança, criando uma base tecnológica mais robusta para auditoria, geração de métricas e expansão futura. Dessa forma, o projeto contribui para profissionalizar a gestão da ONG, ampliar seu impacto social e fortalecer a prestação de contas para patrocinadores, voluntários e sociedade.

FUNCIONALIDADES IMPLEMENTADAS

Endereço

- Criar endereço com validação da API ViaCep
- Listar todos os endereços
- Buscar endereço por ID
- Filtrar por cidade (lógica de negócio)
- Atualizar endereço – também conta com validação da API ViaCep
- Excluir endereço

Dentista

- Criar dentista com validação de sexo/categoria
- Listar todos os dentistas
- Filtrar dentistas por cidade (lógica: relação com Endereco)
- Listar apenas os dentistas disponíveis (lógica: filtro boolean)
- Buscar dentista por CPF
- Atualizar dados do dentista
- Excluir dentista

Colaborador

- Criar colaborador

- Listar todos os colaboradores
- Buscar por CPF
- Atualizar dados do colaborador
- Excluir colaborador

Pedido de Ajuda

- Criar pedido com validação de CPF
- Listar todos os pedidos
- Buscar por CPF
- Buscar por ID
- Filtrar pedidos por data específica
- Atualizar status e atribuir dentista, para permitir aprovação/rejeição
- Excluir pedido

Beneficiário

- Criar beneficiário - recebe os dados inseridos no PedidoAjuda
- Listar todos os beneficiários
- Filtrar beneficiários por programa social (lógica: relação com ProgramaSocial)
- Filtrar beneficiários por cidade (lógica: relação com Endereco)
- Buscar por CPF
- Atualizar dados
- Excluir beneficiário

Atendimento

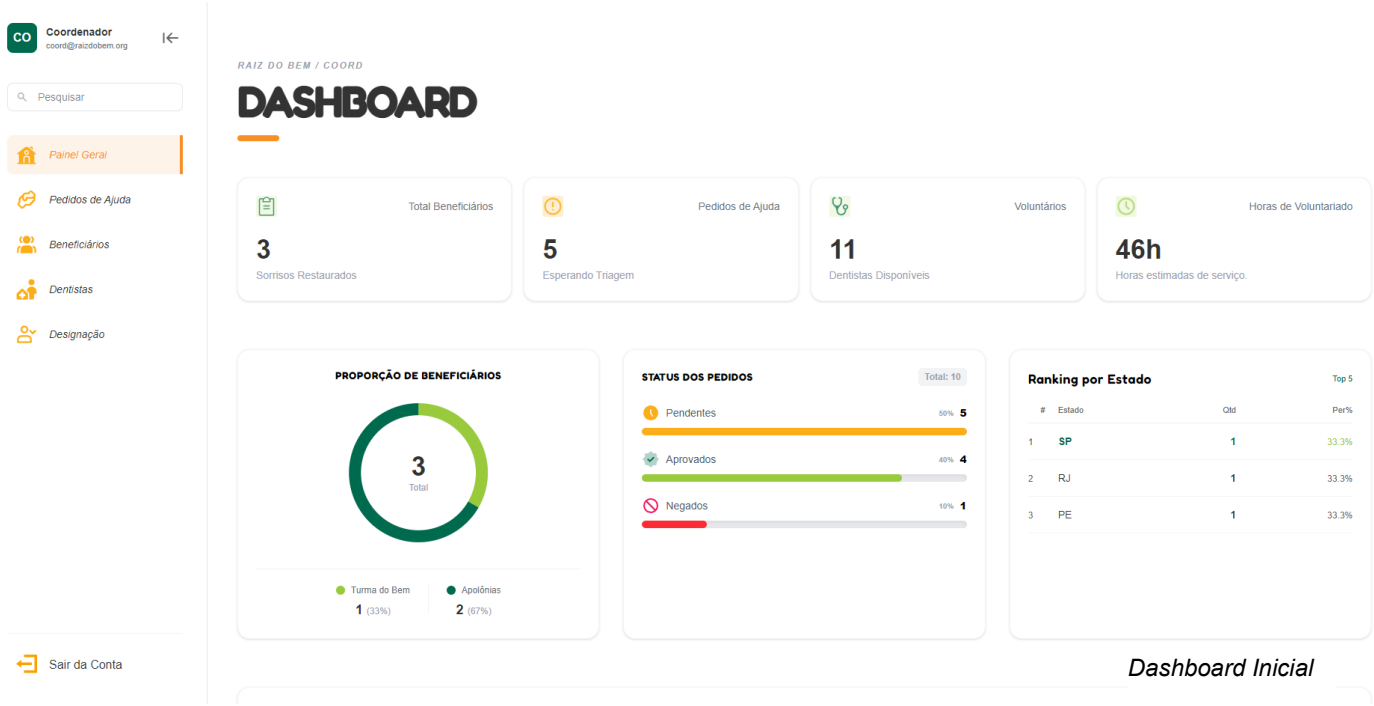
- Criar atendimento
- Listar todos os atendimentos
- Buscar por CPF do beneficiário - JOIN com Beneficiario
- Busca por ID
- Finalizar atendimento – atualizando dados com prontuário, colaborador responsável que relata, e atualizando data final.

Módulos de Apoio

- Listar catálogos fixos
- Validar existência por ID para uso em fluxos de negócio
- Sem CRUD completo por decisão de domínio (dados de referência)

VISUALIZAÇÃO DE TELAS DO SISTEMA

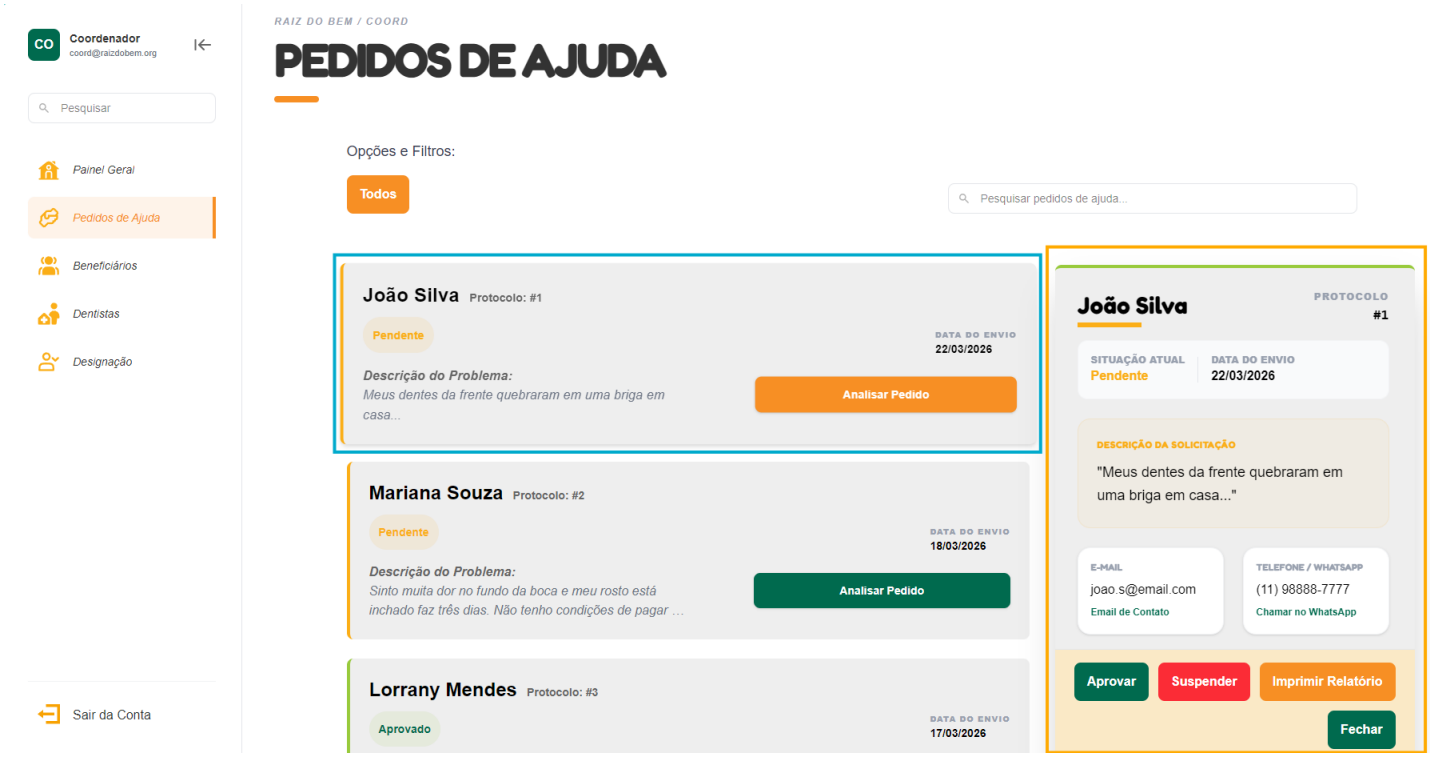
Painel Geral



Dashboard Inicial

Tela inicial do nosso sistema, com uma visão geral, que nos permite analisar informações sobre beneficiários, pedidos de ajuda, voluntários, e gráficos da ong.

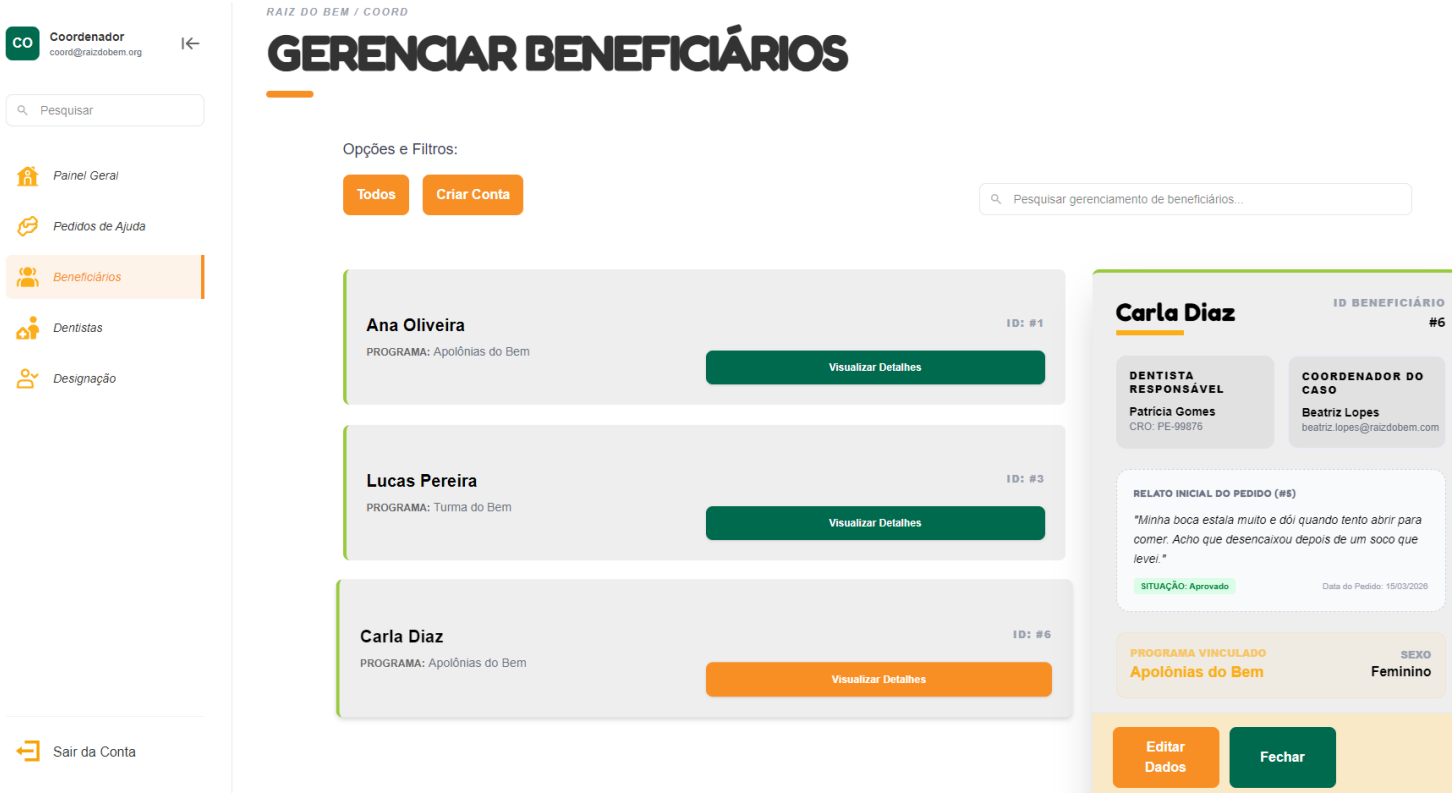
Pedidos de Ajuda



Análise de Pedido de Ajuda

Nessa segunda tela, já implementada, é possível visualizar todos os pedidos de ajuda encontrados. Clicando no botão de analisar pedido, aparecem as possibilidades de Aprovar/Rejeitar o pedido de ajuda, e a opção de imprimir um relatório desse pedido de ajuda.

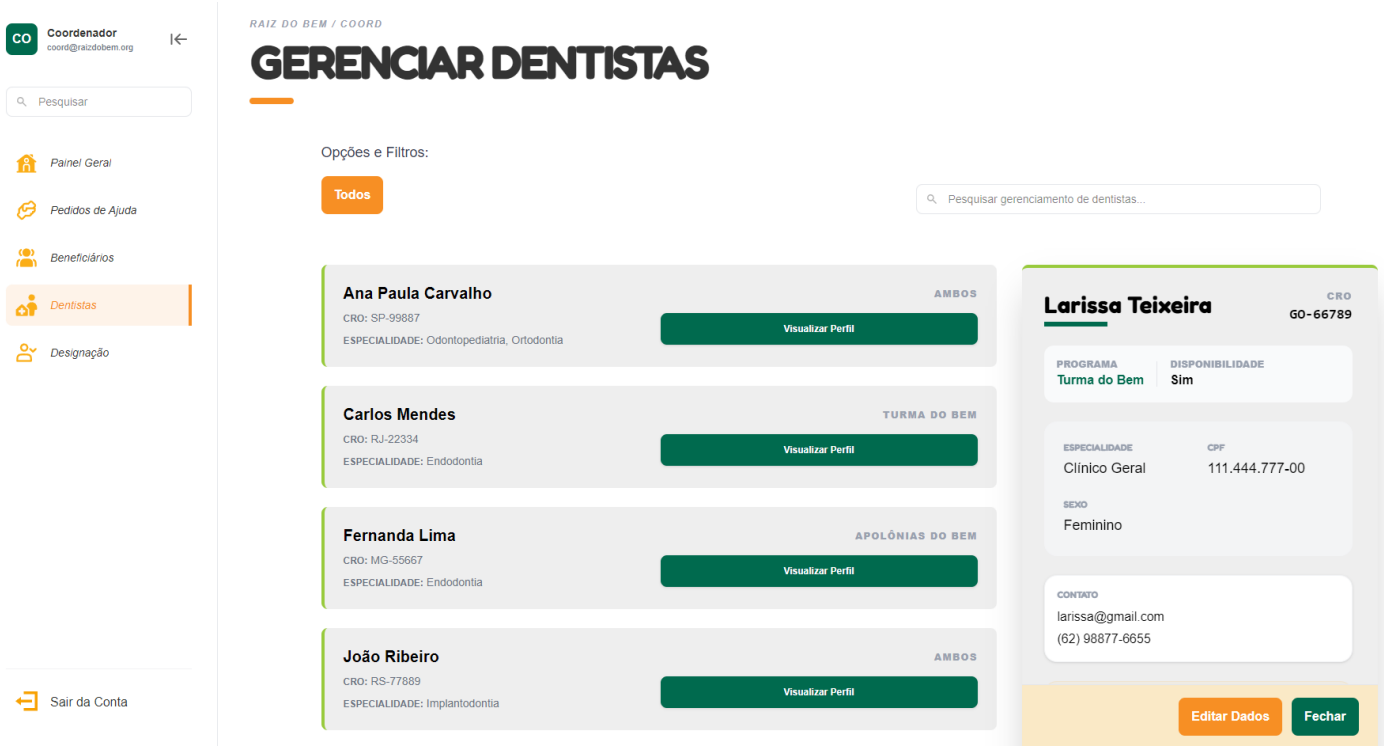
Beneficiários



Verificação geral dos beneficiários

Nessa tela de beneficiários, pode se ver, todas as informações do beneficiário que está no sistema da ONG. Incluindo informações sobre o seu atendimento e dentista responsável pelo caso.

Dentistas



Informações gerais dos dentistas

Nesta seção, são exibidas todas as informações sobre os dentistas incluindo seus dados, especialidades, e informações sobre o seu local de atendimento.

MÉTODOS COM LÓGICA DE NEGÓCIO

1. **Método validarEndereco -> EnderecoBO**

```
public Endereco validarEndereco(String cep, String numero, TipoEndereco tipoEndereco){
        ViaCep enderecoBuscado = bo.buscarInformacoesEndereco(cep);

        if(enderecoBuscado == null || enderecoBuscado.isErro()){
            throw new RuntimeException("Endereço não encontrado!!!");
        }
        return new Endereco(
                enderecoBuscado.getLogradouro(),
                cep,
                numero,
                enderecoBuscado.getBairro(),
                enderecoBuscado.getLocalidade(),
                enderecoBuscado.getUf(),
                tipoEndereco
        );
    }
```

Trecho de código do validarEndereco

Valida o cep do endereço, com integração com API do ViaCep. Após digitação do usuário, o método busca pelo cep, e se for válido, são preenchidas todas as informações de logradouro, bairro, cidade e estado, faltando ao usuário reponder apenas o número do local, e o tipo, se é residencial ou profissional, o que economiza tempo na experiência do usuário.

2. **Método validarCpf -> Validacao**

```
public boolean validarCpf(String cpf) {
        return cpf != null && cpf.matches("\\d{11}");
    }
```

Validação do Cpf

Esse método, criado para validar Cpf, garante que o valor inserido será atendido em caso específico, e todas as classes que a utilizem terão garantidos 11 caracteres numéricos na inserção do campo para validação firme, já que conta com uso de Regex. Garante que em caso de digitação incorreta, é necessário reinserir dado. Método reutilizável, que foi implementado em uma classe geral, para permitir a "herança" e o uso por outras classes.

3. **Método validarCep -> EnderecoBO**

```
public boolean validarCep(String cep){
        return cep!=null && cep.matches("\\d{8}");
    }
```

Valida entrada do Cep

Método semelhante ao de validação de Cpf, neste caso, também é utilizado o regex, para validação de campo, numérico, e neste caso, com 8 diígitos, padrão brasileiro para o documento de Cpf. Este método permite nova digitação em inputs, caso seja digitado incorretamente, garantindo a entrada de dados desse tipo.

4. Método validarGerarBeneficiario -> PedidoAjudaBO

```java
public void validarGerarBeneficiario(int id, PedidoAjuda novoPedido, int idPrograma){
        PedidoAjuda pedidoAjuda = dao.buscarPorId(id);

        if(pedidoAjuda == null){
            throw new RuntimeException("Pedido não encontrado!!!");
        }
        dao.atualizarPedido(id, novoPedido);

        BeneficiarioBO bo = new BeneficiarioBO();
        bo.adicionar(pedidoAjuda.getIdPedido(), idPrograma);
    }
```

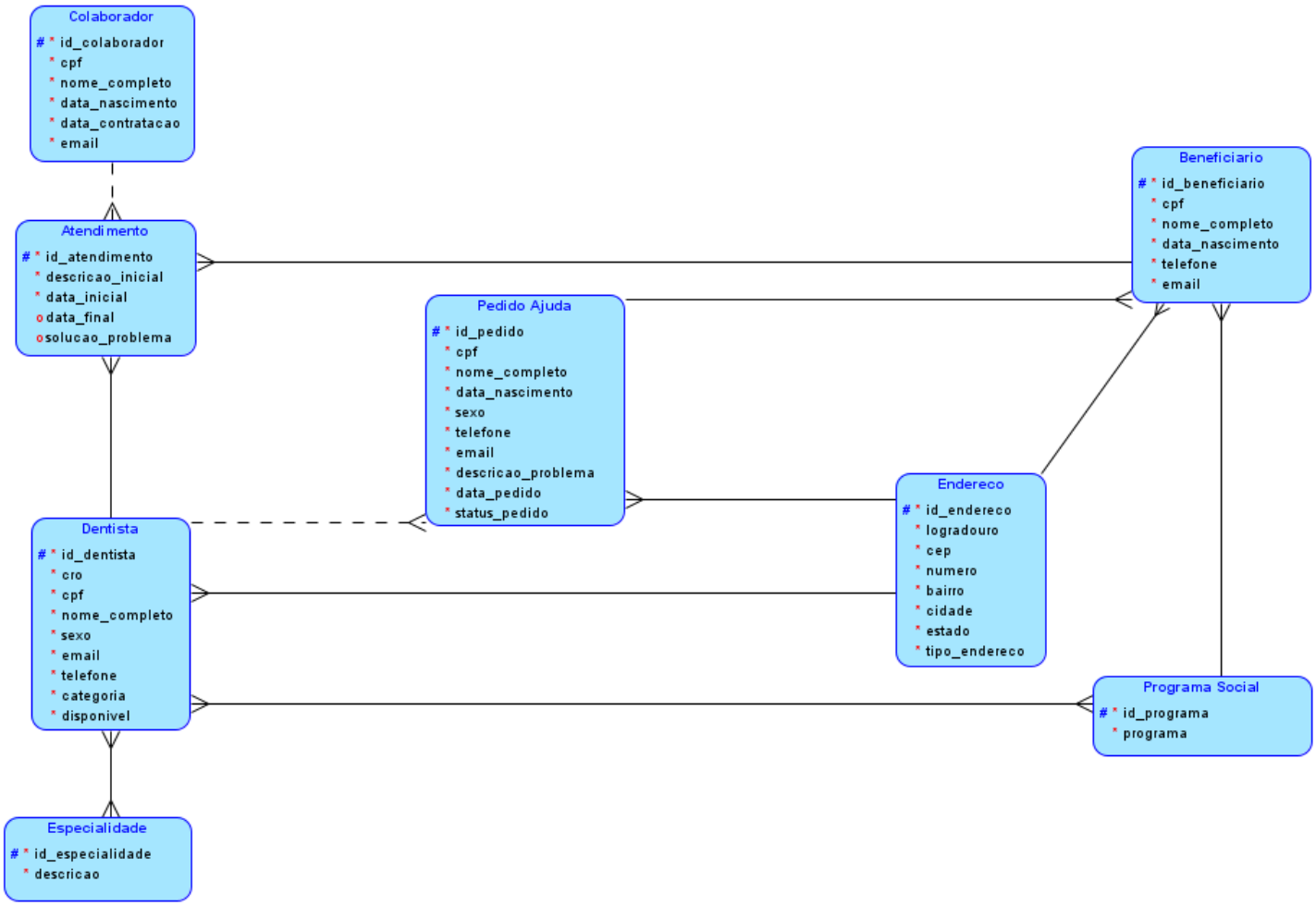
Atualiza o status e gera beneficiário automaticamente

Com este quarto método de regra de negócio, o validarGerarBeneficiario, é possível meio que automatizar a criação de um beneficiario, onde, caso um pedido de ajuda seja atualizado com o status de “APROVADO”, esse método, é implementado, e reduz o processo manual de criação, antes em dois passos – atualização de pedido de ajuda + criação de novo beneficiário, - agora, conta com o preenchimento automático de informações capturadas previamente. Validação essa, que é fundamental, e garante a otimização de fluxo da ONG.

Na controller, este trecho de código exemplifica, o filtro, onde se for identificado que o status pós-atualização de pedido de ajuda é “APROVADO”, segue pelo fluxo que cria o beneficiário. Caso não, seria o “REJEITADO”, que apenas atualiza o pedido, e elimina a posibilidade de ser gerado um beneficiário a partir desse pedido.

```java
if(StatusPedido.APROVADO.equals(status)){
    int idPrograma = view.inputPrograma();
    bo.validarGerarBeneficiario(id, pedido, idPrograma);
}
else{
  bo.atualizar(id, pedido);
}
```

MODELO DE ENTIDADE RELACIONAMENTO – (MER)



Modelo de Entidade Relacionamento, o MLR gerado no Oracle DB, que serve de base para o nosso sistema, e exemplifica, como ficam as relações das entidades no banco de dados. As entidades do banco de dados são:

Pessoas

- Beneficiario
- Dentista
- Colaborador

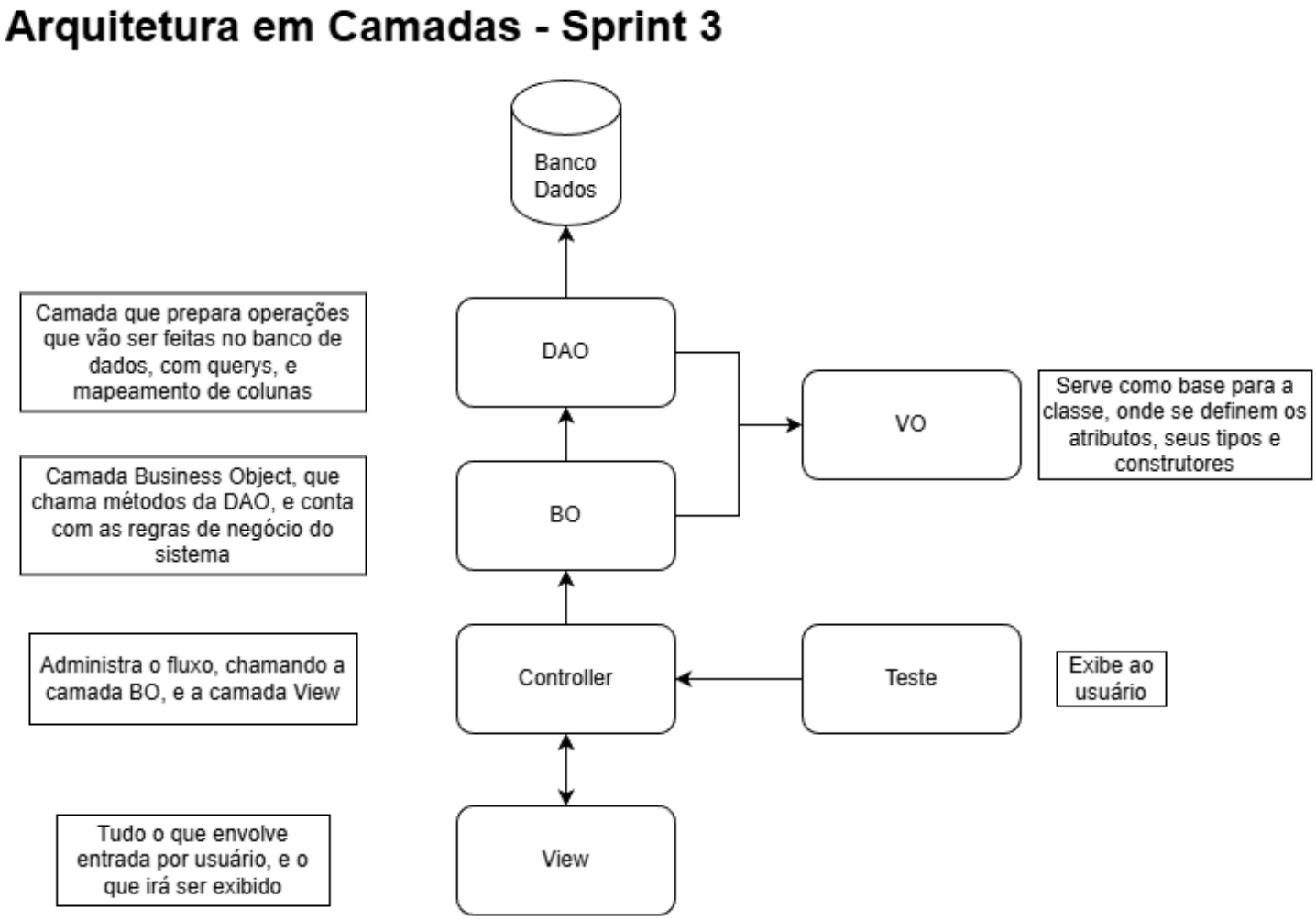
Processos

- Pedido_ajuda
- Atendimento
- Endereço

Apoio

- Especialidade
- Programa_social

ARQUITETURA DO PROJETO



Nossa arquitetura foi desenhada para evitar acoplamento. A camada de View é totalmente isolada do banco de dados, ela só conhece a Controller. As regras de negócio ficam centralizadas em cada BO, enquanto toda a complexidade do SQL e do JDBC puro ficam nas suas respectivas DAOs. A VO, parte da Model atua como nossa estrutura de dados comum, permitindo que a informação caminhe de forma organizada em todas as camadas.

DIAGRAMAS DE CLASSES

Diagrama de Classes Simples

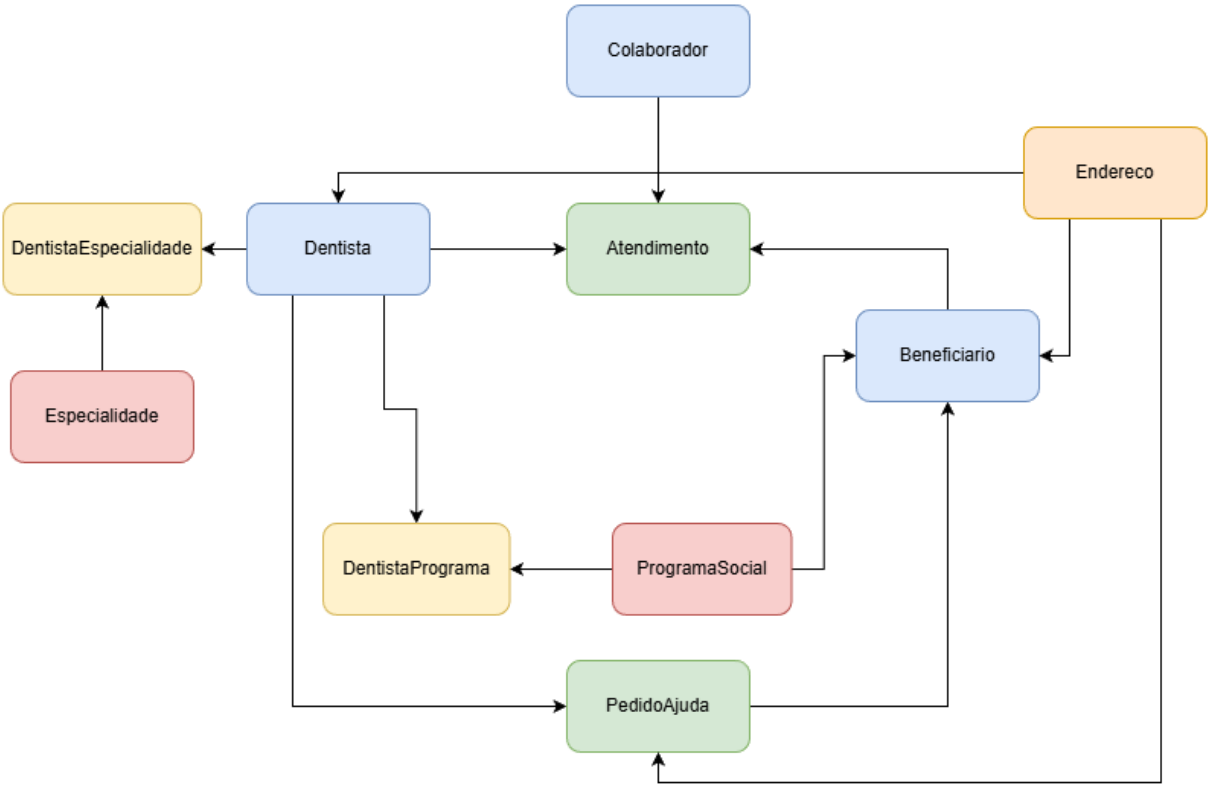


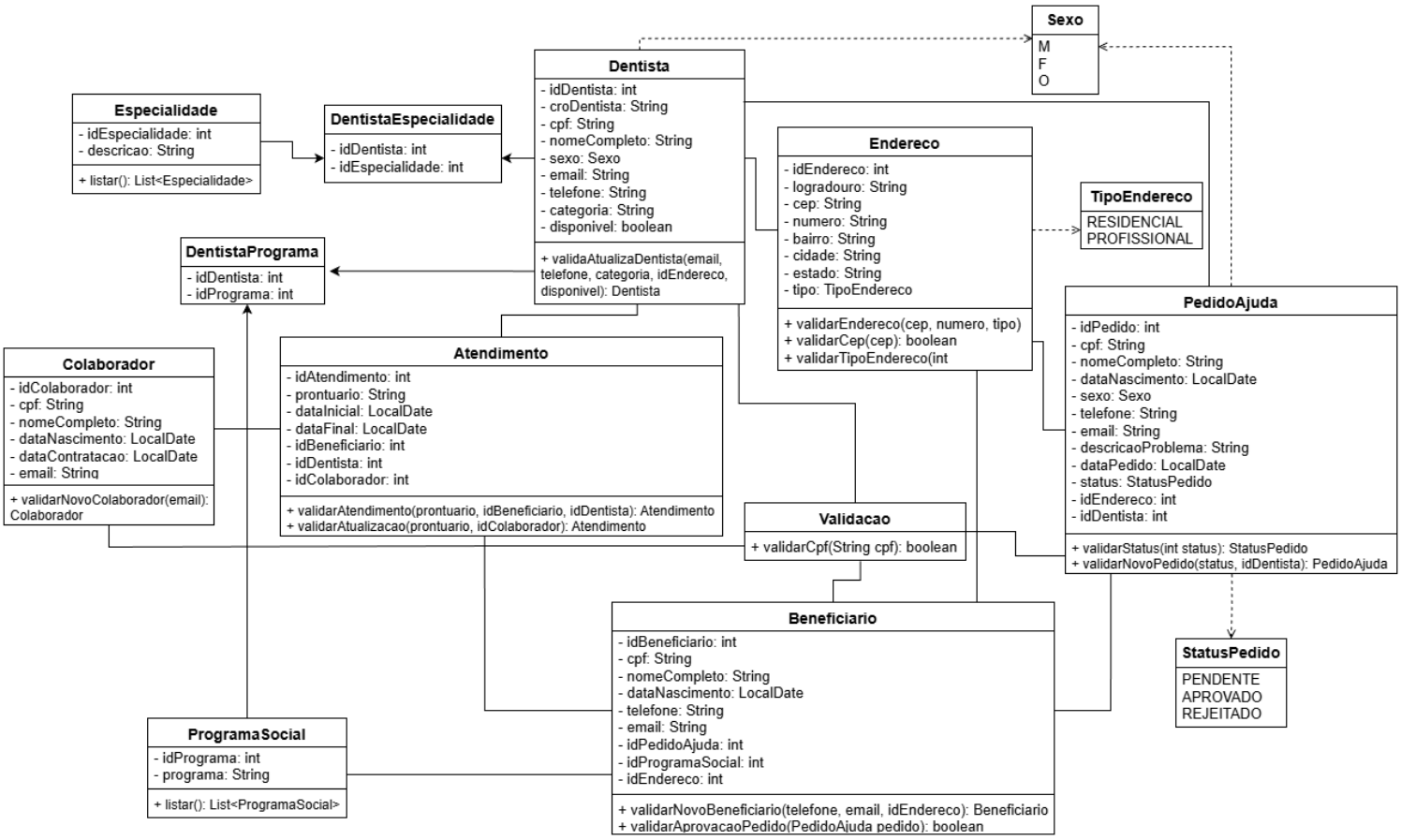
Diagrama mais simples, explicando relações das principais Classe do projeto, e como elas se ligam, refletindo o banco de dados.

Classes presentes no diagrama são:

- Colaborador
- Beneficiario
- Dentista
- PedidoAjuda
- Atendimento
- Endereco
- ProgramaSocial
- DentistaPrograma
- DentistaEspecialidade

Diagrama do Fluxo do Projeto

Diagrama UML



PROCEDIMENTOS PARA EXECUÇÃO

Ferramentas Utilizadas

- **IDE**: Eclipse / NetBeans / IntelliJ IDEA
- **Banco de Dados**: Oracle DB
- **Driver**: JDBC Oracle

Versões Necessárias

- Java JDK 17.0.0 ou superior
- Oracle JDBC Driver 19.0 ou superior
- Gson 2.8.9 (para parsing JSON ViaCep)

Passo a Passo

1) Configurar Banco de Dados

Script de criação das tabelas do banco de dados:

```
--Apagando as tabelas iniciais para não haver conflito
DROP TABLE Dentista_Especialidade CASCADE CONSTRAINTS;
DROP TABLE Dentista_Programa_Social CASCADE CONSTRAINTS;
DROP TABLE Atendimento CASCADE CONSTRAINTS;
DROP TABLE Beneficiario CASCADE CONSTRAINTS;
```

```
DROP TABLE Pedido_Ajuda CASCADE CONSTRAINTS;
DROP TABLE Colaborador CASCADE CONSTRAINTS;
DROP TABLE Dentista CASCADE CONSTRAINTS;
DROP TABLE Endereco CASCADE CONSTRAINTS;
DROP TABLE Especialidade CASCADE CONSTRAINTS;
DROP TABLE Programa_Social CASCADE CONSTRAINTS;

--Criando todas as tabelas
CREATE TABLE Atendimento (
    id_atendimento    NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT
NULL,
    prontuario VARCHAR2 (500 CHAR) NOT NULL,
    data_inicial      DATE NOT NULL,
    data_final        DATE NULL,
    id_beneficiario   NUMBER NOT NULL,
    id_dentista       NUMBER NOT NULL,
    id_colaborador    NUMBER NULL);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_pk PRIMARY KEY (id_atendimento);

CREATE TABLE Beneficiario (
    id_beneficiario   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT
NULL,
    cpf               CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo     VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento   DATE NOT NULL,
    telefone          VARCHAR2 (15 CHAR) NOT NULL,
    email             VARCHAR2 (40 CHAR) NOT NULL,
    id_programa_social NUMBER NOT NULL,
    id_pedido_ajuda   NUMBER NOT NULL,
    id_endereco       NUMBER NOT NULL);
ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_pk PRIMARY KEY (id_beneficiario);

CREATE TABLE Colaborador (
    id_colaborador    NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT
NULL,
    cpf               CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo     VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento   DATE NOT NULL,
    data_contratacao  DATE NOT NULL,
    email             VARCHAR2 (40 CHAR) NOT NULL);
ALTER TABLE Colaborador ADD CONSTRAINT colaborador_pk PRIMARY KEY (id_colaborador);

CREATE TABLE Dentista (
    id dentista   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT NULL,
    cro           VARCHAR2 (10 CHAR) NOT NULL UNIQUE,
    cpf           CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo VARCHAR2 (50 CHAR) NOT NULL,
    sexo          CHAR (1 CHAR) NOT NULL CHECK(sexo IN ('M', 'F', 'O')),
    email         VARCHAR2 (40 CHAR) NOT NULL,
    telefone      VARCHAR2 (15 CHAR) NOT NULL,
    categoria     VARCHAR2 (11 CHAR) NOT NULL CHECK(categoria IN ('COORDENADOR', 'CLINICO')),
    id_endereco   NUMBER NOT NULL,
    disponivel    CHAR (1 CHAR) NOT NULL CHECK(disponivel IN ('S', 'N')));
ALTER TABLE Dentista ADD CONSTRAINT dentista_pk PRIMARY KEY (id_dentista);

CREATE TABLE Dentista Especialidade (
    id_especialidade NUMBER NOT NULL,
    id_dentista      NUMBER NOT NULL);
ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_pk PRIMARY KEY (id_especialidade,
id_dentista);

CREATE TABLE Dentista_Programa_Social (
    id_dentista NUMBER NOT NULL,
    id_programa NUMBER NOT NULL);
ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_pk PRIMARY KEY
(id_dentista, id_programa);

CREATE TABLE Endereco (
    id_endereco   NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT NULL,
    logradouro    VARCHAR2 (50 CHAR) NOT NULL,
    cep           CHAR (8 CHAR) NOT NULL,
    numero        VARCHAR2 (6 CHAR) NOT NULL,
    bairro        VARCHAR2 (40 CHAR) NOT NULL,
    cidade        VARCHAR2 (30 CHAR) NOT NULL,
    estado        VARCHAR2 (30 CHAR) NOT NULL,
```

```
    tipo_endereco VARCHAR2 (12 CHAR) NOT NULL CHECK(tipo_endereco IN ('RESIDENCIAL',
'PROFISSIONAL')));
ALTER TABLE Endereco ADD CONSTRAINT endereco_pk PRIMARY KEY (id_endereco);

CREATE TABLE Especialidade (
    id_especialidade NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT
NULL,
    descricao        VARCHAR2 (25 CHAR) NOT NULL);
ALTER TABLE Especialidade ADD CONSTRAINT especialidade_pk PRIMARY KEY (id_especialidade);

CREATE TABLE Pedido_Ajuda (
    id_pedido          NUMBER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) NOT
NULL,
    cpf                CHAR (11 CHAR) NOT NULL UNIQUE,
    nome_completo      VARCHAR2 (50 CHAR) NOT NULL,
    data_nascimento    DATE NOT NULL,
    sexo               CHAR (1 CHAR) NOT NULL CHECK(sexo IN ('M', 'F', 'O')),
    telefone           VARCHAR2 (15 CHAR) NOT NULL,
    email              VARCHAR2 (40 CHAR) NOT NULL,
    descricao_problema VARCHAR2 (300 CHAR) NOT NULL,
    data_pedido        DATE NOT NULL,
    status_pedido      VARCHAR2 (20 CHAR) NOT NULL CHECK(status_pedido IN ('PENDENTE',
'APROVADO', 'REJEITADO')),
    id_dentista        NUMBER NULL,
    id_endereco        NUMBER NOT NULL);
ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_pk PRIMARY KEY (id_pedido);

CREATE TABLE Programa_Social (
    id_programa NUMBER NOT NULL,
    programa    VARCHAR2 (30 CHAR) NOT NULL);
ALTER TABLE Programa_Social ADD CONSTRAINT programa_social_pk PRIMARY KEY (id_programa);
ALTER TABLE Atendimento ADD CONSTRAINT atendimento_beneficiario_fk FOREIGN KEY
(id_beneficiario) REFERENCES Beneficiario (id_beneficiario);

ALTER TABLE Atendimento ADD CONSTRAINT atendimento_colaborador_fk FOREIGN KEY (id_colaborador)
REFERENCES Colaborador (id_colaborador);

ALTER TABLE Atendimento ADD CONSTRAINT atendimento_dentista_fk FOREIGN KEY (id_dentista)
REFERENCES Dentista (id_dentista);

ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_endereco_fk FOREIGN KEY (id_endereco)
REFERENCES Endereco (id_endereco);

ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_pedido_fk FOREIGN KEY (id_pedido_ajuda)
REFERENCES Pedido_Ajuda (id_pedido);

ALTER TABLE Beneficiario ADD CONSTRAINT beneficiario_programa_fk FOREIGN KEY
(id_programa_social) REFERENCES Programa_Social (id_programa);

ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_den_fk FOREIGN KEY (id_dentista)
REFERENCES Dentista (id_dentista);

ALTER TABLE Dentista_Especialidade ADD CONSTRAINT dent_espec_espec_fk FOREIGN KEY
(id_especialidade) REFERENCES Especialidade (id_especialidade);

ALTER TABLE Dentista ADD CONSTRAINT dentista_end_fk FOREIGN KEY (id_endereco) REFERENCES
Endereco (id_endereco);

ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_dent_fk FOREIGN KEY
(id_dentista) REFERENCES Dentista (id_dentista);

ALTER TABLE Dentista_Programa_Social ADD CONSTRAINT dentista_programa_prog_fk FOREIGN KEY
(id_programa) REFERENCES Programa_Social (id_programa);

ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_dentista_fk FOREIGN KEY (id_dentista)
REFERENCES Dentista (id_dentista);

ALTER TABLE Pedido_Ajuda ADD CONSTRAINT pedido_ajuda_endereco_fk FOREIGN KEY (id_endereco)
REFERENCES Endereco (id_endereco);
```

2) Configurando a conexão com o banco de dados

Editar o arquivo config.properties, definindo a url, usuário e senha:

```
db.driver=oracle.jdbc.OracleDriver
db.url=jdbc:oracle:thin:@SEU_IP:1521:XE
db.user=seu_usuario
db.password=sua_senha
```

PEQUENA DEMONSTRAÇÃO DO PROGRAMA

REFERÊNCIAS RELEVANTES NO PROJETO

Repositório do projeto no GitHub: RaizDoBem-Backend-Java
Site da nossa iniciativa – Raiz do Bem: Raiz do Bem
Caso deseje fazer login no nosso site:

Email: coord@raizdobem.org
Senha: 123

CONCLUSÃO